

More Random Sampling Methods

Peter Occil

This version of the document is dated 2026-07-30.

Peter Occil

1 Contents

- Contents
 - About This Document
 - Specific Distributions
 - * Normal (Gaussian) Distribution
 - * Gamma Distribution
 - * Beta Distribution
 - * Uniform Partition with a Positive Sum
 - * Noncentral Hypergeometric Distributions
 - * von Mises Distribution
 - * Stable Distribution
 - * Phase-Type Distributions
 - * Multivariate Normal (Multinormal) Distribution
 - * Gaussian and Other Copulas
 - * Multivariate Phase-Type Distributions
- Notes
- Appendix
 - Exact, Error-Bounded, and Approximate Algorithms
- License

1.1 About This Document

This is an open-source document; for an updated version, see the source code¹ or its rendering on GitHub². You can send comments on this document on the GitHub issues page³.

My audience for this article is **computer programmers with mathematics knowledge, but little or no familiarity with calculus**.

I encourage readers to implement any of the algorithms given in this page, and report their implementation experiences. In particular, **I seek comments on the following aspects**⁴:

- Are the algorithms in this article (in conjunction with “**Randomization and Sampling Methods**⁵”) easy to implement? Is each algorithm written so that someone could write code for that algorithm after reading the article?

¹<https://github.com/peteroupc/peteroupc.github.io/raw/master/randomnotes.md>

²<https://github.com/peteroupc/peteroupc.github.io/blob/master/randomnotes.md>

³<https://github.com/peteroupc/peteroupc.github.io/issues>

⁴<https://github.com/peteroupc/peteroupc.github.io/issues/18>

⁵<https://peteroupc.github.io/randomfunc.html>

- Does this article have errors that should be corrected?
- Are there ways to make this article more useful to the target audience?

Comments on other aspects of this document are welcome.

1.2 Specific Distributions

Requires random real numbers. This section shows algorithms to sample several popular nonuniform distributions. The algorithms are exact unless otherwise noted, and applications should choose algorithms with either no error (including rounding error) or a user-settable error bound. See the **appendix** for more information.

1.2.1 Normal (Gaussian) Distribution

The *normal distribution*⁶ (also called the Gaussian distribution) takes the following two parameters:

- **mu** () is the mean (average), or where the peak of the distribution’s “bell curve” is.
- **sigma** (σ), the standard deviation, affects how wide the “bell curve” appears. The probability that a number sampled from the normal distribution will be within one standard deviation from the mean is about 68.3%; within two standard deviations (2 times **sigma**), about 95.4%; and within three standard deviations, about 99.7%. (Some publications give σ^2 , or variance, rather than standard deviation, as the second parameter. In this case, the standard deviation is the variance’s square root.)

There are a number of methods for sampling the normal distribution. An application can combine some or all of these.

1. The ratio-of-uniforms method (given as `NormalRatioOfUniforms` below).
2. In the *Box-Muller transformation*, `mu + radius * cos(angle)` and `mu + radius * sin(angle)`, where `angle = RNDRANGEMinMaxExc(0, 2 * pi)` and `radius = sqrt(Expo(0.5)) * sigma`, are two independent values sampled from the normal distribution. The polar method (given as `NormalPolar` below) likewise produces two independent values sampled from that distribution at a time.
3. Karney’s algorithm to sample from the normal distribution, in a manner that minimizes approximation error and without using floating-point numbers (Karney 2016)⁷.

For surveys of Gaussian samplers, see (Thomas et al. 2007)⁸, and (Malik and Hemani 2016)⁹.

```
METHOD NormalRatioOfUniforms(mu, sigma)
  while true
    a=RNDRANGEMinMaxExc(0,1)
    bv = sqrt(2.0/exp(1.0))
    // Or bv = 858/1000.0, which is also correct
    b=RNDRANGEMinMaxExc(0,bv)
    if b*b <= -a * a * 4 * ln(a)
      return (RNDINT(1) * 2 - 1) *
        (b * sigma / a) + mu
    end
  end
END METHOD
```

⁶https://en.wikipedia.org/wiki/Normal_distribution

⁷Karney, C.F.F., 2016. Sampling exactly from the normal distribution. *ACM Transactions on Mathematical Software (TOMS)*, 42(1), pp.1-14. Also: “**Sampling exactly from the normal distribution**”, arXiv:1303.6257v2 [physics.comp-ph], 2014. <https://arxiv.org/abs/1303.6257v2>

⁸Thomas, D., et al., “Gaussian Random Number Generators”, *ACM Computing Surveys* 39(4), 2007.

⁹Malik, J.S., Hemani, A., “Gaussian random number generation: A survey on hardware architectures”, *ACM Computing Surveys* 49(3), 2016.

```

METHOD NormalPolar(mu, sigma)
  while true
    a = RNDRANGEMinMaxExc(0,1)
    b = RNDRANGEMinMaxExc(0,1)
    if RNDINT(1) == 0: a = 0 - a
    if RNDINT(1) == 0: b = 0 - b
    c = a * a + b * b
    if c != 0 and c <= 1
      c = sqrt(-ln(c) * 2 / c)
      return [a * sigma * c + mu, b * sigma * c + mu]
    end
  end
END METHOD

```

Notes:

1. The *standard normal distribution* is implemented as `Normal(0, 1)`.
2. Methods implementing a variant of the normal distribution, the *discrete Gaussian distribution*, generate *integers* that closely follow the normal distribution. Examples include the one in (Karney 2016)¹⁰, an improved version in (Du et al. 2021)¹¹, as well as so-called “constant-time” methods such as (Micciancio and Walter 2017)¹² that are used above all in *lattice-based cryptography*.
3. The following are some approximations to the normal distribution that papers have suggested:
 - The sum of twelve `RNDRANGEMinMaxExc(0, sigma)` numbers, subtracted by $6 * \text{sigma}$, to generate an approximate normal variate with mean 0 and standard deviation `sigma`. (Kabal 2000/2019)¹³ “warps” this sum in the following way (before adding the mean `mu`) to approximate the normal distribution better: `ssq = sum * sum;`
`sum = (((0.0000001141*ssq - 0.0000005102) * ssq + 0.00007474) * ssq +`
`0.0039439) * ssq + 0.98746) * sum.` See also “**Irwin–Hall distribution**”¹⁴, namely the sum of `n` many `RNDRANGEMinMaxExc(0, 1)` numbers, on Wikipedia. D. Thomas (2014)¹⁵, describes a more general approximation called CLTk, which combines `k` numbers in `[0, 1]` sampled from the uniform distribution as follows: `RNDRANGEMinMaxExc(0, 1) - RNDRANGEMinMaxExc(0, 1) + RNDRANGEMinMaxExc(0, 1) - ....`
 - Numerical **inversions** of the normal distribution’s cumulative distribution function (CDF, or the probability of getting `X` or less at random), including those by Wichura, by Acklam, and by Luu (Luu 2016)¹⁶. See also “**A literate program to compute the inverse of the normal CDF**”¹⁷.
4. A pair of *q-Gaussian* random variates with parameter `q` less than 3 can be generated using the Box–Muller transformation, except `radius` is `radius=sqrt(-2*(pow(u,1-qp)-1)/(1-qp))`

¹⁰Karney, C.F.F., 2016. Sampling exactly from the normal distribution. ACM Transactions on Mathematical Software (TOMS), 42(1), pp.1-14. Also: “**Sampling exactly from the normal distribution**”, arXiv:1303.6257v2 [physics.comp-ph], 2014. <https://arxiv.org/abs/1303.6257v2>

¹¹Du, Yusong, Baoying Fan, and Baodian Wei. “An improved exact sampling algorithm for the standard normal distribution.” Computational Statistics (2021): 1-17, also arXiv:2008.03855 [cs.DS].

¹²Micciancio, D. and Walter, M., “Gaussian sampling over the integers: Efficient, generic, constant-time”, in Annual International Cryptology Conference, August 2017 (pp. 455-485).

¹³Kabal, P., “Generating Gaussian Pseudo-Random Variates”, McGill University, 2000/2019.

¹⁴https://en.wikipedia.org/wiki/Irwin%E2%80%93Hall_distribution

¹⁵Thomas, D.B., 2014, May. FPGA Gaussian random number generators with guaranteed statistical accuracy. In *2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines* (pp. 149-156).

¹⁶Luu, T., “Fast and Accurate Parallel Computation of Quantile Functions for Random Number Generation”, Dissertation, University College London, 2016.

¹⁷https://www.johndcook.com/blog/normal_cdf_inverse/

(where $qp=(1+q)/(3-q)$ and $u=\text{RNDRANGEMinMaxExc}(0, 1)$), and the two variates are not statistically independent (Thistleton et al. 2007)¹⁸.

5. A well-known result says that adding n many $\text{Normal}(0, 1)$ variates, and dividing by $\sqrt{n}$, results in a new $\text{Normal}(0, 1)$ variate.

1.2.2 Gamma Distribution

The following method samples a number from a *gamma distribution* and is based on Marsaglia and Tsang’s method from 2000¹⁹ and (Liu et al. 2015)²⁰. Usually, the number expresses either—

- the lifetime (in days, hours, or other fixed units) of a random component with an average lifetime of `meanLifetime`, or
- a random amount of time (in days, hours, or other fixed units) that passes until as many events as `meanLifetime` happen.

Here, `meanLifetime` must be an integer or noninteger greater than 0.

```
METHOD GammaDist(meanLifetime)
  // Needs to be greater than 0
  if meanLifetime <= 0: return error
  // Exponential distribution special case if
  // `meanLifetime` is 1 (see also (Devroye 1986), p. 405)
  if meanLifetime == 1: return Expo(1)
  if meanLifetime < 0.3 // Liu, Martin, Syring 2015
    lamda = (1.0/meanLifetime) - 1
    w = meanLifetime / (1-meanLifetime) * exp(1)
    r = 1.0/(1+w)
    while true
      z = 0
      x = RNDRANGEMinMaxExc(0, 1)
      if x <= r: z = -ln(x/r)
      else: z = -Expo(lamda)
      ret = exp(-z/meanLifetime)
      eta = 0
      if z>=0: eta=exp(-z)
      else: eta=w*lamda*exp(lamda*z)
      if RNDRANGEMinMaxExc(0, eta) < exp(-ret-z): return ret
    end
  end
  d = meanLifetime
  v = 0
  if meanLifetime < 1: d = d + 1
  d = d - (1.0 / 3) // NOTE: 1.0 / 3 must be a fractional number
  c = 1.0 / sqrt(9 * d)
  while true
    x = 0
    while true
      x = Normal(0, 1)
```

¹⁸Thistleton, W., Marsh, J., et al., “Generalized Box-Müller Method for Generating q-Gaussian Random Deviates”, *IEEE Transactions on Information Theory* 53(12), 2007.

¹⁹Marsaglia, G., Tsang, W.W., “A simple method for generating gamma variables”, *ACM Transactions on Mathematical Software* 26(3), 2000.

²⁰Liu, C., Martin, R., Syring, N., “Simulating from a gamma distribution with small shape parameter”, arXiv:1302.1884v3 [stat.CO], 2015. <https://arxiv.org/abs/1302.1884v3>

```

        v = c * x + 1;
        v = v * v * v
        if v > 0: break
    end
    u = RNDRANGEMinMaxExc(0,1)
    x2 = x * x
    if u < 1 - (0.0331 * x2 * x2): break
    if ln(u) < (0.5 * x2) + (d * (1 - v + ln(v))): break
end
ret = d * v
if meanLifetime < 1
    ret = ret * pow(RNDRANGEMinMaxExc(0, 1), 1.0 / meanLifetime)
end
return ret
END METHOD

```

Notes:

1. The following is a useful identity for the gamma distribution: $\text{GammaDist}(a) = \text{BetaDist}(a, b - a) * \text{GammaDist}(b)$ (Stuart 1962)²¹.
2. The gamma distribution is usually defined to have a second parameter (called **theta** here), which is unfortunately defined differently in different works. For example, the gamma variate can be either multiplied or divided by **theta** depending on the work.
3. For other algorithms to sample from the gamma distribution, see Luengo (2022)²².

Example: Moment exponential distribution (Dara and Ahmad 2012): $\text{GammaDist}(2)*\text{beta}$ (or $(\text{Expo}(1)+\text{Expo}(1))*\text{beta}$), where $\text{beta} > 0$.

1.2.3 Beta Distribution

The beta distribution takes on values on the interval (0, 1). Its two parameters, **a** and **b**, are both greater than 0 and describe the distribution's shape. Depending on **a** and **b**, the shape can be a smooth peak or a smooth valley.

The following method samples a number from a *beta distribution*, in the interval [0, 1).

```

METHOD BetaDist(a, b)
    if b==1 and a==1: return RNDRANGEMinMaxExc(0, 1)
    // Min-of-uniform
    if a==1: return 1.0-pow(RNDRANGEMinMaxExc(0, 1),1.0/b)
    // Max-of-uniform. Use only if a is small to
    // avoid accuracy problems, as pointed out
    // by Devroye 1986, p. 675.
    if b==1 and a < 10: return pow(RNDRANGEMinMaxExc(0, 1),1.0/a)
    x=GammaDist(a)
    return x/(x+GammaDist(b))
END METHOD

```

I give an **error-bounded sampler**²³ for the beta distribution (when **a** and **b** are both 1 or greater) in a separate page.

²¹A. Stuart, "Gamma-distributed products of independent random variables", *Biometrika* 49, 1962.

²²Luengo, E.A., "Gamma Pseudo Random Number Generators", *ACM Computing Surveys*, 2022. <https://dl.acm.org/doi/abs/10.1145/3527157>

²³<https://peteroupc.github.io/exporand.html>

1.2.4 Uniform Partition with a Positive Sum

The following algorithm chooses at random a uniform partition of the number `sum` into `n` parts, and returns an `n`-item list of the chosen numbers, which sum to `sum` assuming no rounding error. In this algorithm, `n` must be an integer greater than 0, and `sum` must be greater than 0. The method was described in Bini and Buttazzo (2005)²⁴ and Mai et al. (2022)²⁵.

```
METHOD UniformSum(n, sum):
  if n<=0 or sum<=0: return error
  w=1; nn=n-1;ret=NewList()
  while nn>0
    v=w*(1-pow(RNDU01MinMaxExc(),1.0/nn))
    ret.append(v*sum)
    w=w-v; nn=nn-1
  end
  AddItem(ret, w*sum); return ret
END METHOD
```

1.2.5 Noncentral Hypergeometric Distributions

The following variants of the hypergeometric distribution are described in detail by Agner Fog in “**Biased Urn Theory**”²⁶.

Let there be m balls that each have one of two or more colors. For each color, assign each ball of that color the same weight (a real number 0 or greater). Then:

1. **Wallenius’s hypergeometric distribution:** Choose one ball not yet chosen, with probability equal to its weight divided by the sum of weights of balls not yet chosen. Repeat until exactly n items are chosen this way. Then for each color, count the number of items of that color chosen this way.
2. **Fisher’s hypergeometric distribution:** For each ball, choose it with probability equal to its weight divided by the sum of weights of all balls. (Thus, each ball is independently chosen or not chosen depending on its weight.) If exactly n items were chosen this way, stop. Otherwise, start over. Then among the last n items chosen this way, count the number of items of each color.

For both distributions, if there are two colors, there are four parameters: m , $ones$, n , $weight$, such that—

- for the first color, there are $ones$ many balls each with weight $weight$;
- for the second color, there are $(m - ones)$ many balls each with weight 1; and
- the random variate is the number of chosen balls of the first color.

1.2.6 von Mises Distribution

The *von Mises distribution* describes a distribution of circular angles and uses two parameters: `mean` is the mean angle and `kappa` is a shape parameter. The distribution is uniform at `kappa` = 0 and approaches a normal distribution with increasing `kappa`.

The algorithm below samples a number from the von Mises distribution, and is based on the Best–Fisher algorithm from 1979 (as described in (Devroye 1986)²⁷ with errata incorporated).

```
METHOD VonMises(mean, kappa)
```

²⁴Bini, B., Buttazzo, G.C., “Measuring the Performance of Schedulability Tests”, *Real-Time Systems* 30, 129-154 (2005)

²⁵Mai, J., Craig, J.R., Tolson, B.A., “The pie-sharing problem: Unbiased sampling of $N+1$ summative weights”, *Environmental Modelling & Software* 148, February 2022. <https://www.sciencedirect.com/science/article/pii/S1364815221003248>

²⁶<https://cran.r-project.org/web/packages/BiasedUrn/vignettes/UrnTheory.pdf>

²⁷Devroye, L., *Non-Uniform Random Variate Generation*, 1986.

```

if kappa < 0: return error
if kappa == 0
    return RNDRANGEMinMaxExc(mean-pi, mean+pi)
end
r = 1.0 + sqrt(4 * kappa * kappa + 1)
rho = (r - sqrt(2 * r)) / (kappa * 2)
s = (1 + rho * rho) / (2 * rho)
while true
    u = RNDRANGEMinMaxExc(-pi, pi)
    v = RNDRANGEMinMaxExc(0, 1)
    z = cos(u)
    w = (1 + s*z) / (s + z)
    y = kappa * (s - w)
    if y*(2 - y) - v >= 0 or ln(y / v) + 1 - y >= 0
        if angle < -1: angle = -1
        if angle > 1: angle = 1
        // NOTE: Inverse cosine replaced here
        // with `atan2` equivalent
        angle = atan2(sqrt(1-w*w), w)
        if u < 0: angle = -angle
        return mean + angle
    end
end
END METHOD

```

1.2.7 Stable Distribution

As more and more numbers, sampled independently at random in the same way, are added together, their distribution tends to a *stable distribution*²⁸, which resembles a curve with a single peak, but with generally “fatter” tails than the normal distribution. (Here, the stable distribution means the “alpha-stable distribution”.) The pseudocode below uses the Chambers–Mallows–Stuck algorithm. The **Stable** method, implemented below, takes two parameters:

- **alpha** is a stability index in the interval (0, 2].
- **beta** is an asymmetry parameter in the interval [-1, 1]; if **beta** is 0, the curve is symmetric.

```

METHOD Stable(alpha, beta)
    if alpha <= 0 or alpha > 2: return error
    if beta < -1 or beta > 1: return error
    halfpi = pi * 0.5
    unif=RNDRANGEMinMaxExc(-halfpi, halfpi)
    c=cos(unif)
    expo=Expo(1)
    if alpha == 1
        s=sin(unif)
        if beta == 0: return s/c
        return 2.0*((unif*beta+halfpi)*s/c -
            beta * ln(halfpi*expo*c/(unif*beta+halfpi)))/pi
    else
        z=-tan(alpha*halfpi)*beta
    end
END METHOD

```

²⁸https://en.wikipedia.org/wiki/Stable_distribution

```

        ug=unif+atan2(-z, 1)/alpha
        cpow=pow(c, -1.0 / alpha)
        return pow(1.0+z*z, 1.0 / (2*alpha))*
            (sin(alpha*ug)*cpow)*
            pow(cos(unif-alpha*ug)/expo, (1.0 - alpha) / alpha)
    end
END METHOD

```

Methods implementing the strictly geometric stable and general geometric stable distributions are shown next (Kozubowski 2000)²⁹. Here, `alpha` is in $(0, 2]$, `lamda` is greater than 0, and `tau`'s absolute value is not more than $\min(1, 2/\alpha - 1)$. The result of `GeometricStable` is a symmetric Linnik distribution if `tau` = 0, or a Mittag-Leffler distribution if `tau` = 1 and `alpha` < 1.

```

METHOD GeometricStable(alpha, lamda, tau)
    rho = alpha*(1-tau)/2
    sign = -1
    if tau==1 or RNDINT(1)==0 or RNDRANGEMinMaxExc(0, 1) < tau
        rho = alpha*(1+tau)/2
        sign = 1
    end
    w = 1
    if rho != 1
        rho = rho * pi
        cotparam = RNDRANGEMinMaxExc(0, rho)
        w = sin(rho)*cos(cotparam)/sin(cotparam)-cos(rho)
    end
    return Expo(1) * sign * pow(lamda*w, 1.0/alpha)
END METHOD

```

```

METHOD GeneralGeoStable(alpha, beta, mu, sigma)
    z = Expo(1)
    if alpha == 1: return mu*z+Stable(alpha, beta)*sigma*z+
        sigma*z*beta*2*pi*ln(sigma*z)
    else: return mu*z+
        Stable(alpha, beta)*sigma*pow(z, 1.0/alpha)
END METHOD

```

1.2.8 Phase-Type Distributions

A *phase-type distribution* models a sum of exponential random variates driven by a **Markov chain**³⁰. The Markov chain has `n` normal states and one “absorbing” or terminating state. This distribution has two parameters:

- `alpha`, an `n`-item array showing the probability of starting the chain at each normal state.
- `s`, an `n`×`n` *subgenerator matrix*, a list of `n` lists of `n` values each. The values in each list (each normal state of the Markov chain) must sum to 0 or less, and for each state `i`, `s[i][i]` is 0 minus the rate of that state’s exponential random variate, and each entry `s[i][j]` with `i`≠`j` is the relative probability for moving to state `j`.

The method `PhaseType`, given later, samples from a phase-type distribution given the two parameters

²⁹Tomasz J. Kozubowski, “Computer simulation of geometric stable distributions”, *Journal of Computational and Applied Mathematics* 116(2), pp. 221-229, 2000. [https://doi.org/10.1016/S0377-0427\(99\)00318-0](https://doi.org/10.1016/S0377-0427(99)00318-0) <https://www.sciencedirect.com/science/article/pii/S0377042799003180>

³⁰<https://peteroupc.github.io/randomnotes.html>

above. (The pseudocode assumes each number in `alpha` and `s` is a rational number, because it uses `NormalizeRatios`.)

```

METHOD GenToTrans(s)
  // Converts a subgenerator matrix to a
  // more intuitive transition matrix.
  m=[];
  for j in 0...size(s)
    m[j]=[]; for i in 0...size(s)+1: AddItem(m[j],0)
  end
  for i in 0...size(s)
    isum=Sum(s[i])
    if isum<0: m[i][size(s)]=isum/s[i][i]
    for j in 0...size(s)
      if j!=i: m[i][j]=-s[i][j]/s[i][i]
    end
  end
  return m
END METHOD

METHOD PhaseType(alpha, s)
  // Setup
  trans=GenToTrans(s)
  // Sampling
  state=WeightedChoice(NormalizeRatios(alpha))
  ret=0
  while state<size(s)
    ret=ret+Expo(-s[state][state])
    state=WeightedChoice(NormalizeRatios(trans[state]))
  end
  return ret
END METHOD

```

Note: An **inhomogeneous phase-type** random variate has the form $G(\text{PhaseType}(\alpha, s))$, where $G(x)$ is a function designed to control the heaviness of the distribution’s tail (Bladt 2021)³¹. For example, $G(x) = \text{pow}(x, 1.0/\text{beta})$, where $\text{beta}>0$, leads to a tail as heavy as a Weibull distribution.

1.2.9 Multivariate Normal (Multinormal) Distribution

The following pseudocode generates a random vector (list of numbers) that follows a ***multivariate normal (multinormal) distribution***³². The method `MultivariateNormal` takes the following parameters:

- A list, `mu ( )`, which indicates the means to add to the random vector’s components. `mu` can be `nothing`, in which case each component will have a mean of zero.
- A list of lists `cov`, that specifies a *covariance matrix* (Σ), a symmetric positive definite $N \times N$ matrix, where N is the number of components of the random vector. (An $N \times N$ matrix is *positive definite* if its determinant [overall scale] is greater than 0 and if either the matrix is 1×1 or a smaller matrix formed by removing the last row and column is positive definite.)

³¹Bladt, Martin. “Phase-type distributions for claim severity regression modeling.” ASTIN Bulletin: The Journal of the IAA (2021): 1-32.

³²https://en.wikipedia.org/wiki/Multivariate_normal_distribution

```

METHOD Decompose(matrix)
  numrows = size(matrix)
  if size(matrix[0])!=numrows: return error
  // Does a Cholesky decomposition of a matrix
  // assuming it's positive definite and invertible
  ret=NewList()
  for i in 0...numrows
    submat = NewList()
    for j in 0...numrows: AddItem(submat, 0)
    AddItem(ret, submat)
  end
  s1 = sqrt(matrix[0][0])
  if s1==0: return ret // For robustness
  for i in 0...numrows
    ret[0][i]=matrix[0][i]*1.0/s1
  end
  for i in 0...numrows
    msum=0.0
    for j in 0...i: msum = msum + ret[j][i]*ret[j][i]
    sq=matrix[i][i]-msum
    if sq<0: sq=0 // For robustness
    ret[i][i]=math.sqrt(sq)
  end
  for j in 0...numrows
    for i in (j + 1)...numrows
      // For robustness
      if ret[j][j]==0: ret[j][i]=0
      if ret[j][j]!=0
        msum=0
        for k in 0...j: msum = msum + ret[k][i]*ret[k][j]
        ret[j][i]=(matrix[j][i]-msum)*1.0/ret[j][j]
      end
    end
  end
  return ret
END METHOD

METHOD VecAdd(a, b)
  c=[]; for j in 0...size(a): c[j]=a[j]+b[j]
  return c
END METHOD

METHOD VecScale(a, scalar)
  c=[]; for j in 0...size(a): c[j]=a[j]*scalar
  return c
END METHOD

METHOD MultivariateNormal(mu, cov)
  vars=NewList()
  for j in 0...mulen: AddItem(vars, Normal(0, 1))
  return MultivariateCov(mu,cov,vars)
END METHOD

```

```

METHOD MultivariateCov(mu, cov, vars)
  // Returns mu + cov^(1/2)*vars
  mulen=size(cov)
  if mu != nothing
    mulen = size(mu)
    if mulen!=size(cov): return error
    if mulen!=size(cov[0]): return error
  end
  // NOTE: If multiple random points will
  // be generated using the same covariance
  // matrix, an implementation can consider
  // precalculating the decomposed matrix
  // in advance rather than calculating it here.
  cho=Decompose(cov)
  i=0
  ret=NewList()
  while i<mulen
    msum = 0
    for j in 0...mulen: msum=cho[j][i]*vars[j]
    AddItem(ret, msum)
    i=i+1
  end
  if mu!=nothing: ret=VecAdd(ret, mu)
  return ret
end

```

Note: The **Python sample code**³³ contains a variant of this method for generating multiple random vectors in one call.

Examples:

1. A vector that follows a **binormal distribution** (two-variable multinormal distribution) is a vector of two numbers from the normal distribution, and can be sampled using the following idiom: `MultivariateNormal([mu1, mu2], [[s1*s1, s1*s2*rho], [rho*s1*s2, s2*s2]])`, where `mu1` and `mu2` are the means of the vector's two components, `s1` and `s2` are their standard deviations, and `rho` is a *correlation coefficient* greater than -1 and less than 1 (0 means no correlation).
2. **Log-multinormal distribution:** Generate a multinormal random vector, then apply `exp(n)` to each component `n`.
3. A **Beckmann distribution:** Generate a random binormal vector `vec`, then apply `PNorm(vec, 2)` to that vector. (`PNorm` is given in the main page's section "**Random Points on a Sphere**"³⁴.)
4. A **Rice (Rician) distribution** is a Beckmann distribution in which the binormal random pair is generated with `m1 = m2 = a / sqrt(2)`, `rho = 0`, and `s1 = s2 = b`, where `a` and `b` are the parameters to the Rice distribution.
5. **Rice-Norton distribution:** Generate `vec = MultivariateNormal([v,v], [[w,0,0], [0,w,0],[0,0,w]])` (where `v = a/sqrt(m*2)`, `w = b*b/m`, and `a`, `b`, and `m` are the parameters to the Rice-Norton distribution), then apply `PNorm(vec, 2)` to that vector.

³³<https://peteroupc.github.io/randomgen.zip>

³⁴https://peteroupc.github.io/randomnotes.md#Random_Points_on_a_Sphere

6. A **standard complex normal distribution**³⁵ is a binormal distribution in which the binormal random pair is generated with $s_1 = s_2 = \text{sqrt}(0.5)$ and $\mu_1 = \mu_2 = 0$ and treated as the real and imaginary parts of a complex number.
7. **Multivariate Linnik distribution**: Generate a multinormal random vector, then multiply each component by x , where $x = \text{GeometricStable}(\alpha/2.0, 1, 1)$, where α is a parameter in $(0, 2]$ (Kozubowski 2000)³⁶.
8. **Multivariate exponential power distribution** (Solaro 2004)³⁷: `MultivariateCov(mu, cov, vec)`, where `vec = RandomPointOnSphere(m, pow(Gamma(m/s,1)*2,1.0/s), 2)`, m is the dimension, $s > 0$ is a shape parameter, μ is the mean as an m -dimensional vector (m -item list), and `cov` is a covariance matrix.
9. **Elliptical distribution**: `MultivariateCov(mu, cov, RandomPointOnSphere(dims, z, 2))`, where z is an arbitrary random variate, dims is the number of dimensions, μ is a dims -dimensional location vector, and `cov` is a $\text{dims} \times \text{dims}$ covariance matrix. See, for example, Fang et al. (1990)³⁸.
10. **Mean-variance mixture of normal distributions** (Barndorff-Nielsen et al. 1982)³⁹: `VecAdd(mu, VecAdd(VecScale(delta, v), VecScale(MultivariateNormal(nothing, cov), sqrt(z))))`, where μ and delta are n -dimensional vectors, `cov` is a covariance matrix, and v is an arbitrary random variate 0 or greater.
11. **Mean mixture of normal distributions** (Bhagwat and Marchand 2022)⁴⁰: `MultivariateNormal(VecAdd(theta, cov))` where theta is an n -dimensional location vector, a is an n -dimensional “perturbation vector”, `cov` is a covariance matrix, and v is an arbitrary random variate.

1.2.10 Gaussian and Other Copulas

A *copula* is a way to describe the dependence between randomly sampled numbers.

One example is a *Gaussian copula*; this copula is sampled by sampling from a **multinormal distribution**, then converting the resulting numbers to *dependent* uniform random values. In the following pseudocode, which implements a Gaussian copula:

- The parameter `covar` is the covariance matrix for the multinormal distribution.
- `erf(v)` is the **error function**⁴¹ of the number v .

```
METHOD GaussianCopula(covar)
  mvn=MultivariateNormal(nothing, covar)
  for i in 0...size(covar)
    // Apply the normal distribution's CDF
    // to get uniform numbers
    mvn[i] = (erf(mvn[i]/(sqrt(2)*sqrt(covar[i][i])))+1)*0.5
  end
  return mvn
END METHOD
```

³⁵https://en.wikipedia.org/wiki/Complex_normal_distribution

³⁶Tomasz J. Kozubowski, “Computer simulation of geometric stable distributions”, *Journal of Computational and Applied Mathematics* 116(2), pp. 221-229, 2000. [https://doi.org/10.1016/S0377-0427\(99\)00318-0](https://doi.org/10.1016/S0377-0427(99)00318-0) <https://www.sciencedirect.com/science/article/pii/S0377042799003180>

³⁷Giammatteo, P., and Di Mascio, T., “Wilson-Hilferty-type approximation for Poisson Random Variable”, *Advances in Science, Technology and Engineering Systems Journal* 5(2), 2020.

³⁸Fang et al., *Symmetric multivariate and related distributions*, 1990.

³⁹Malik, J.S., Hemani, A., “Gaussian random number generation: A survey on hardware architectures”, *ACM Computing Surveys* 49(3), 2016.

⁴⁰Du, Yusong, Baoying Fan, and Baodian Wei. “An improved exact sampling algorithm for the standard normal distribution.” *Computational Statistics* (2021): 1-17, also arXiv:2008.03855 [cs.DS].

⁴¹https://en.wikipedia.org/wiki/Error_function

Each of the resulting uniform random values will be in the interval $[0, 1]$, and each one can be further transformed to any other probability distribution (which is called a *marginal distribution* or *marginal* here) by taking the quantile of that uniform number for that distribution (see “**Inverse Transform Sampling**”⁴², and see also (Cario and Nelson 1997)⁴³.)

Note: The Gaussian copula is also known as the *normal-to-anything* method.

Examples:

1. To generate two correlated uniform random values with a Gaussian copula, generate `GaussianCopula([[1, rho], [rho, 1]])`, where `rho` is the Pearson correlation coefficient, in the interval $[-1, 1]$. (Other correlation coefficients besides `rho` exist. For example, for a two-variable Gaussian copula, the **Spearman correlation coefficient**⁴⁴ `srho` can be converted to `rho` by `rho = sin(srho * pi / 6) * 2`. Other correlation coefficients, and other measures of dependence between randomly sampled numbers, are not further discussed in this document.)
2. The following example generates a two-dimensional random vector that follows a Gaussian copula with exponential marginals (`rho` is the Pearson correlation coefficient, and `rate1` and `rate2` are the rates of the two exponential marginals).

```
METHOD CorrelatedExpo(rho, rate1, rate2)
  copula = GaussianCopula([[1, rho], [rho, 1]])
  // Transform to exponentials using that
  // distribution's quantile function
  return [-log1p(-copula[0]) / rate1,
          -log1p(-copula[1]) / rate2]
END METHOD
```

3. The *T-Poisson hierarchy* (Knudson et al. 2021)⁴⁵ is a way to generate N-dimensional Poisson-distributed random vectors via copulas. Each of the N dimensions is associated with—
 - a parameter `lamda`, and
 - a marginal distribution that may not be discrete and takes on only nonnegative values.

To sample from the **T-Poisson** hierarchy—

1. sample an N-dimensional random vector via a copula (such as `GaussianCopula`), producing an N-dimensional vector of correlated uniform numbers; then
2. for each component in the vector, replace it with that component's quantile for the corresponding marginal; then
3. for each component in the vector, replace it with `Poisson(lamda * c)`, where `c` is that component and `lamda` is the `lamda` parameter for the corresponding dimension.

The following example implements the T-Poisson hierarchy using a Gaussian copula and exponential marginals.

```
METHOD PoissonH(rho, rate1, rate2, lambda1, lambda2)
  vec = CorrelatedExpo(rho, rate1, rate2)
  return [Poisson(lambda1*vec[0]), Poisson(lambda2*vec[1])]
END METHOD
```

⁴²https://peteroupc.github.io/randomfunc.html#Inverse_Transform_Sampling

⁴³Cario, M. C., B. L. Nelson, “Modeling and generating random vectors with arbitrary marginal distributions and correlation matrix”, 1997.

⁴⁴https://en.wikipedia.org/wiki/Rank_correlation

⁴⁵Knudson, A.D., Kozubowski, T.J., et al., “A flexible multivariate model for high-dimensional correlated count data”, *Journal of Statistical Distributions and Applications* 8:6, 2021.

Other kinds of copulas describe different kinds of dependence between randomly sampled numbers. Examples of other copulas are—

- the **Fréchet–Hoeffding upper bound copula** $[x, x, \dots, x]$ (for example, $[x, x]$), where $x = \text{RNDRANGEMinMaxExc}(0, 1)$,
- the **Fréchet–Hoeffding lower bound copula** $[x, 1.0 - x]$ where $x = \text{RNDRANGEMinMaxExc}(0, 1)$,
- the **product copula**, where each number is a separately generated $\text{RNDRANGEMinMaxExc}(0, 1)$ (indicating no dependence between the numbers), and
- the **Archimedean copulas**, described by M. Hofert and M. Mächler (2011)⁴⁶.

1.2.11 Multivariate Phase-Type Distributions

The following pseudocode generates a random vector (of d coordinates) following a *multivariate phase-type distribution* called MPH*. In addition to parameters **alpha** and **s**, there is also a *reward matrix* **r**, such that $r[i][j]$ is the probability of adding to coordinate j when state i is visited. (The pseudocode assumes each number in **alpha**, **s**, and **r** is a rational number, because it uses `NormalizeRatios`.)

```
METHOD MPH(alpha, s, r)
  if len(r[0])<1 or len(r)!=len(s): return error
  // Setup
  trans=GenToTrans(s)
  ret=[]; for i in 0...size(r[0]): AddItem(ret,0)
  // Sampling
  state=WeightedChoice(NormalizeRatios(alpha))
  ret=0
  while state<size(s)
    rs=WeightedChoice(NormalizeRatios(r[state]))
    ret[rs]=ret[rs]+Expo(-s[state][state])
    state=WeightedChoice(NormalizeRatios(trans[state]))
  end
  return ret
END METHOD
```

Note: An inhomogeneous version of MPH* can be as follows: $[G_1(\text{mph}[1]), G_2(\text{mph}[2]), \dots, G_D(\text{mph}[d])]$, where **mph** is a d -dimensional MPH* vector and $G_1, G_2, \dots, G_D$ are strictly increasing functions whose domain and range are the positive real line and whose “slope” is defined on the whole domain (Albrecher et al. 2022)⁴⁷.

2 Notes

3 Appendix

3.1 Exact, Error-Bounded, and Approximate Algorithms

There are three kinds of randomization algorithms:

1. An *exact algorithm* is an algorithm that samples from the exact distribution requested, assuming that computers—

⁴⁶Hofert, M., and Mächler, M. “Nested Archimedean Copulas Meet R: The nacopula Package”. *Journal of Statistical Software* 39(9), 2011, pp. 1-20.

⁴⁷Albrecher, Hansjörg, Mogens Bladt, and Jorge Yslas. “Fitting inhomogeneous phase-type distributions to data: the univariate and the multivariate case.” *Scandinavian Journal of Statistics* 49, no. 1 (2022): 44-77.

- can store and operate on real numbers (which have unlimited precision), and
- can generate independent uniform random real numbers

(Devroye 1986, p. 1-2)⁴⁸. However, an exact algorithm implemented on real-life computers can incur error due to the use of fixed precision (especially floating-point numbers), such as rounding and cancellations. An exact algorithm can achieve a guaranteed bound on accuracy (and thus be an *error-bounded algorithm*) using either arbitrary-precision or interval arithmetic (see also Devroye 1986, p. 2)⁴⁹. All methods given on this page are exact unless otherwise noted. Note that the `RNDRANGEMinMaxExc` method is exact in theory, but has no required implementation.

2. An *error-bounded algorithm* is a sampling algorithm with the following requirements:

- If the ideal distribution is discrete (takes on values that can map to integers and back without loss), the algorithm samples exactly from that distribution. (But see the note later.)
- If the ideal distribution is not discrete, the algorithm samples from a distribution that is close to the ideal within a user-specified error tolerance (see later for details). The algorithm can instead sample a number from the distribution only partially, as long as the fully sampled number can be made close to the ideal within any error tolerance desired.
- In sampling from a distribution, the algorithm incurs no approximation error not already present in the inputs (except errors needed to round the final result to the user-specified error tolerance).

Many error-bounded algorithms use random bits as their only source of randomness. An application should use error-bounded algorithms whenever possible.

Most algorithms on this page, though, are not *error-bounded* when naïvely implemented in most number formats (including floating-point numbers). (There are number formats such as “constructive reals” or “recursive reals” that allow real numbers to be approximated to a user-specified error (Boehm 2020)⁵⁰.)

3. An *inexact*, *approximate*, or *biased algorithm* is any sampling algorithm that is neither exact nor error-bounded. This includes algorithms that sample from a distribution that is close to the desired distribution, but not within a user-specified error tolerance (see also Devroye 1986, p. 2)⁵¹. An application should use this kind of algorithm only if it’s willing to trade accuracy for speed.

There are many ways to describe closeness between two distributions. One suggestion by Devroye and Gravel (2020)⁵² is Wasserstein distance (or “earth-mover distance”), which they proved has a simple definition in terms of the quantile function (Theorem 8). Here, an algorithm has accuracy ϵ (the user-specified error tolerance) if it samples from a distribution that is close to the ideal distribution by a Wasserstein distance of not more than ϵ .

Examples:

1. Sampling from the exponential distribution via `-ln(RNDRANGEMinMaxExc(0, 1))` is an *exact algorithm* (in theory), but not an *error-bounded* one for common floating-point number formats. The same is true of the Box–Muller transformation.
2. Karney’s algorithm for the normal distribution (Karney 2016)⁵³, as well as Karney’s implementation of von Neumann’s exponential distribution sampler (Karney 2016)⁵⁴ are both

⁴⁸Devroye, L., *Non-Uniform Random Variate Generation*, 1986.

⁴⁹Devroye, L., *Non-Uniform Random Variate Generation*, 1986.

⁵⁰Boehm, Hans-J. “Towards an API for the real numbers.” In Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 562-576. 2020.

⁵¹Devroye, L., *Non-Uniform Random Variate Generation*, 1986.

⁵²Devroye, L., Gravel, C., “Random variate generation using only finitely many unbiased, independently and identically distributed random bits”, arXiv:1502.02539v6 [cs.IT], 2020. <https://arxiv.org/abs/1502.02539v6>

⁵³Karney, C.F.F., 2016. Sampling exactly from the normal distribution. ACM Transactions on Mathematical Software (TOMS), 42(1), pp.1-14. Also: “Sampling exactly from the normal distribution”, arXiv:1303.6257v2 [physics.comp-ph], 2014. <https://arxiv.org/abs/1303.6257v2>

⁵⁴Karney, C.F.F., 2016. Sampling exactly from the normal distribution. ACM Transactions on Mathematical Software (TOMS), 42(1), pp.1-14. Also: “Sampling exactly from the normal distribution”, arXiv:1303.6257v2 [physics.comp-ph],

error-bounded, because they return a result that can be made to come close to the normal or exponential distribution, respectively, within any error tolerance desired simply by appending more random digits to the end. See also (Oberhoff 2018)⁵⁵.

3. Examples of *approximate algorithms* include sampling from a Gaussian-like distribution via a sum of `RNDRANGEMinMaxExc(0, 1)`, or most cases of modulo reduction to produce uniform-like integers at random (see notes in the section “**RNDINT**”⁵⁶). The following approximate algorithm for the Poisson distribution is another example (Giammatteo and Di Mascio (2020)⁵⁷): `floor(1.0/3 + pow(max(0, Normal(0, 1))*pow(mean,1/6.0)*2/3 + pow(mean, 2.0/3)), 3.0/2)`, where `mean` is greater than 50.

Note: A discrete distribution can be sampled in finite time on average if and only if its so-called *Shannon entropy* is finite (Knuth and Yao 1976)⁵⁸. Unfortunately, some discrete distributions have infinite Shannon entropy, such as some members of the zeta Dirichlet family of distributions (Devroye and Gravel 2020)⁵⁹. Thus, in practice, an approximate or error-bounded sampler is needed for these distributions. Saad et al. (2020)⁶⁰ discuss how to sample an approximation of a discrete distribution with a user-specified error tolerance, but only if the ideal distribution takes on a finite number of values (and thus has finite Shannon entropy). On the other hand, a distribution has finite Shannon entropy whenever—

- it takes on only integers 1 or greater and has a finite t^{th} moment for some $t > 0$ (“long-run average” of values raised to t^{th} power) (Baccetti and Visser 2013)⁶¹, or as a special case,
- it takes on only integers 1 or greater and has a finite mean (“long-run average”), or
- it has the form $X + n$, where n is a constant and X is a random variate whose distribution has finite Shannon entropy.

4 License

Any copyright to this page is released to the Public Domain. In case this is not possible, this page is also licensed under **Creative Commons Zero**⁶².

2014. <https://arxiv.org/abs/1303.6257v2>

⁵⁵Oberhoff, Sebastian, “**Exact Sampling and Prefix Distributions**”, *Theses and Dissertations*, University of Wisconsin Milwaukee, 2018. <https://dc.uwm.edu/etd/1888>

⁵⁶https://peteroupc.github.io/randomfunc.html#RNDINT_Random_Integers_in_0_N

⁵⁷Giammatteo, P., and Di Mascio, T., “**Wilson-Hilferty-type approximation for Poisson Random Variable**”, *Advances in Science, Technology and Engineering Systems Journal* 5(2), 2020.

⁵⁸Knuth, Donald E. and Andrew Chi-Chih Yao. “**The complexity of nonuniform random number generation**”, in *Algorithms and Complexity: New Directions and Recent Results*, 1976.

⁵⁹Devroye, L., Gravel, C., “**Random variate generation using only finitely many unbiased, independently and identically distributed random bits**”, arXiv:1502.02539v6 [cs.IT], 2020. <https://arxiv.org/abs/1502.02539v6>

⁶⁰Feras A. Saad, Cameron E. Freer, Martin C. Rinard, and Vikash K. Mansinghka, “**Optimal Approximate Sampling From Discrete Probability Distributions**”, arXiv:2001.04555 [cs.DS], also in *Proc. ACM Program. Lang.* 4, POPL, Article 36 (January 2020), 33 pages. <https://arxiv.org/abs/2001.04555>

⁶¹Baccetti, Valentina, and Matt Visser. “**Infinite Shannon entropy.**” *Journal of Statistical Mechanics: Theory and Experiment* 2013, no. 04 (2013): P04010, also in arXiv:1212.5630.

⁶²<https://creativecommons.org/publicdomain/zero/1.0/>